



SMARTFLOW: Intelligent Water Utility Management System

IET ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of
CSA0509 – Database Management Systems

to the award of the degree of
BACHELOR OF TECHNOLOGY
IN
Computer Science And Engineering

Submitted by
Kadhirezhilan. D(192511228)

Under the Supervision of
Dr. R. Kesavan
Professor



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

CHAPTER 1: INTRODUCTION

1.1 Background

Municipal water utilities generate continuous operational information from customer connections, smart meters, billing systems, payment channels, complaint desks, and field operations. A database system must therefore support both transactional workloads and append-heavy telemetry workloads. SMARTFLOW is proposed as a unified platform that places the relational database at the center of these activities.

1.2 Problem Statement

A conventional utility application may store customer details, readings, bills, payments, and complaints without demonstrating how the underlying DBMS handles high-volume data, indexing, concurrency, and transaction safety. The project addresses this gap by designing a system in which major operational actions can be connected directly to DBMS concepts.

- High-volume meter readings require efficient historical range retrieval.
- Latest-reading lookups should not repeatedly scan the telemetry table.
- Concurrent bill generation must not create duplicate invoices.
- Payment updates must preserve account balances under concurrent access.
- Leak analysis should compare recent consumption against a historical baseline.
- Database optimization should be demonstrable through real EXPLAIN plans and measurements.

1.3 Objectives

1. Design a normalized MySQL database for water utility operations.
2. Demonstrate indexed-sequential telemetry organization using B+Tree indexes.
3. Implement an application-level hash table using modulo hashing and separate chaining.
4. Provide transaction-safe billing and payment workflows.
5. Demonstrate concurrency control using row locking and optimistic versioning.
6. Provide Query Lab, Transaction Center, Concurrency Lab, and Performance Lab modules.
7. Produce a structured academic report and evidence plan for evaluation.

1.4 Scope

The proposed scope covers zones, connections, meters, readings, leak intelligence, billing, payments, complaints, audit logs, database labs, reporting, and role-based access. The design includes both operational pages and academic demonstration pages.

1.5 Significance

The significance of SMARTFLOW lies in making DBMS concepts visible through a realistic domain. Instead of treating SQL as a collection of CRUD statements, the project connects schema design, constraints, indexes, transactions, locks, query plans, and data structures to real utility workflows.

1.6 Organization of the Report

Chapters 1–12 progressively cover the problem, literature, requirements, architecture, database design, DBMS concepts, implementation plan, testing, performance evaluation, security, conclusions, and supporting appendices.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Modern utility information systems increasingly depend on centralized relational databases, telemetry platforms, analytics, and secure web interfaces. Database design remains important because operational information must be consistent while telemetry workloads can become very large.

2.2 Utility Management Information Systems

Utility systems typically combine customer account management, metering, billing, payments, service requests, and operational monitoring. These functions produce different access patterns: account data is transactional, while meter readings are sequential and time-oriented.

2.3 Database Management in Utility Systems

Relational DBMS principles such as normalization, primary and foreign keys, constraints, indexing, transactions, and recovery are directly applicable to utility management. SMARTFLOW uses these principles as the foundation of its design.

2.4 Indexing and B+Tree Access

B+Tree indexes are appropriate for equality and range predicates. A composite index beginning with `connection_id` and followed by `reading_timestamp` supports retrieval of a connection's readings over a time window.

2.5 Hashing in Database Applications

Hash tables provide expected constant-time point lookup when the hash distribution is appropriate. SMARTFLOW intentionally keeps the academic hash demonstration at the application layer rather than incorrectly describing a MySQL InnoDB B+Tree index as a hash table.

2.6 Transactions and Concurrency

Transaction processing protects multi-step updates through atomic commit and rollback. Concurrency control is required when two operators access the same bill or complaint simultaneously. SMARTFLOW combines database constraints, row locks, and version tokens.

2.7 Research Gap

The project focuses on integrating several DBMS topics into one coherent demonstrator: telemetry indexing, hashing, leak analytics, billing transactions, concurrency experiments, query-plan visualization, and benchmark evidence.

2.8 Summary

The review establishes the need for a system that is both operationally meaningful and academically demonstrative. SMARTFLOW is designed around this combination.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Existing System

The existing-system model considered in this project represents fragmented utility operations in which customer records, meter readings, billing, complaints, and operational monitoring may be managed separately or without explicit DBMS demonstrations.

3.2 Limitations

- Repeated scans of large telemetry tables.
- Weak visibility into query execution plans.
- Risk of duplicate billing in concurrent workflows.
- Limited demonstration of ACID behavior.
- Manual investigation of abnormal consumption.
- Insufficient traceability of sensitive financial and operational changes.

3.3 Proposed System

SMARTFLOW centralizes utility data and provides a command-center interface backed by MySQL. The system includes operational modules and dedicated DBMS laboratories so that evaluators can inspect both application behavior and database behavior.

SMARTFLOW Three-Layer Architecture

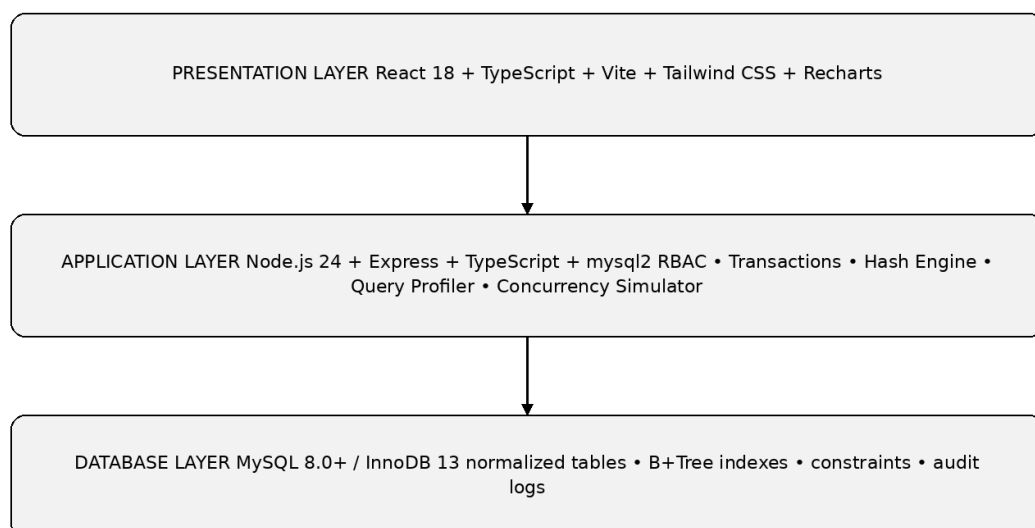


Figure 3.1: Proposed SMARTFLOW three-layer architecture.

3.4 Functional Requirements

Module	Key Functions
Dashboard	KPIs, consumption trends, zone summaries, priority actions
Connections	Search, filters, connection profile, 360° history
Meter Readings	Telemetry stream, historical range queries, status filters
Leak Intelligence	Baseline calculation, deviation scoring, inspection tickets
Billing	Cycle management, tier calculation, duplicate prevention
Payments	Ledger posting, balance reconciliation, receipts
Complaints	Ticket lifecycle, assignment, OCC version checks
Query Lab	Before/after SQL and EXPLAIN analysis
Transaction Center	BEGIN, SAVEPOINT, COMMIT, ROLLBACK demonstrations
Concurrency Lab	Dual-session lock and race-condition experiments
Performance	Latency, row scans, access type, throughput
Reports	CSV/printable reports and audit information

3.5 Non-Functional Requirements

Requirement	Description
Security	Authentication, RBAC, parameterized SQL, audit logging
Performance	Indexed access paths and measurable query latency
Reliability	Atomic transactions and rollback on failure
Scalability	Batch seeding and support for large telemetry datasets
Usability	Clean, data-dense civic-tech interface
Maintainability	Modular backend and frontend structure

3.6 Hardware Requirements

Component	Recommended Requirement
Processor	Intel Core i3 / equivalent or better

RAM	8 GB or above recommended
Storage	At least 10 GB free for development datasets
Display	1366 × 768 or higher
Network	Local network / internet where required for development services

3.7 Software Requirements

Software	Purpose
MySQL 8.0+	Relational database and InnoDB transaction engine
Node.js 24	Backend runtime
TypeScript	Backend/frontend type safety
Express	REST API server
mysql2	Raw parameterized SQL access
React 18 + Vite	Frontend application
Tailwind CSS	UI styling
Recharts	Charts and analytics visuals
Git	Version control

3.8 User Roles

Role	Primary Access
ADMIN	Full system management, seeding, settings, DBMS labs
BILLING_OPERATOR	Billing and payment operations
FIELD_ENGINEER	Meter, leak, and field inspection operations
COMPLAINT_OPERATOR	Complaint assignment and lifecycle management

3.9 System Workflow

SMARTFLOW Operational Workflow

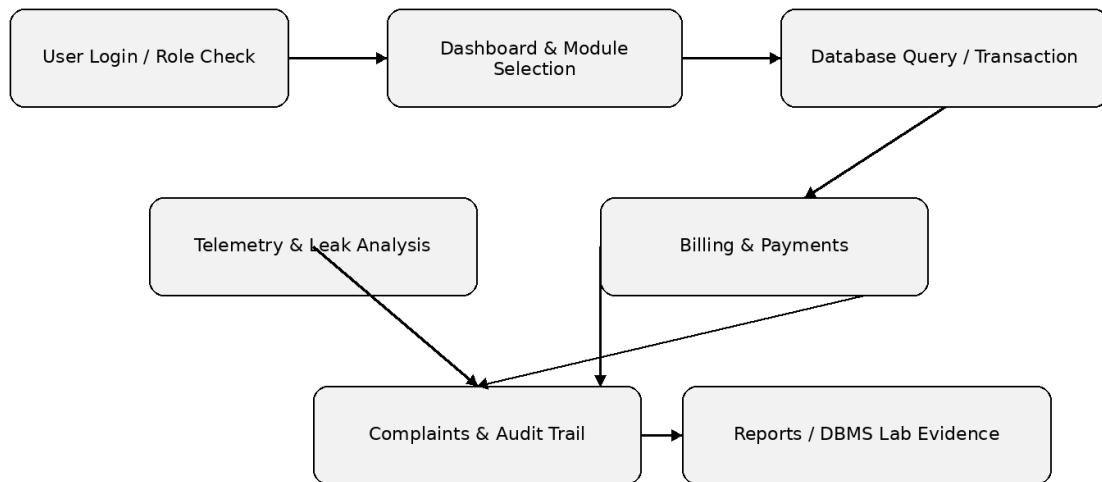


Figure 3.2: High-level SMARTFLOW operational workflow.

3.10 Summary

The analysis establishes the system requirements and shows how each operational need maps to a DBMS mechanism.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

SMARTFLOW follows a presentation–application–database architecture. The frontend provides responsive dashboards and data-dense operational screens. The backend provides REST APIs, transaction orchestration, hashing, profiling, and concurrency simulation. MySQL InnoDB provides persistence, constraints, locking, and recovery.

4.2 Technology Stack

Layer	Technology	Role
Frontend	React 18, TypeScript, Vite	Web interface and interaction
UI	Tailwind CSS, Lucide Icons	Design system and icons
Charts	Recharts	Telemetry and performance visualizations
Backend	Node.js 24, Express, TypeScript	API and business logic
Database Access	mysql2	Prepared SQL and transaction control
Database	MySQL 8.0+, InnoDB	Persistent relational data
Testing / Benchmarking	Node.js scripts	Validation and performance measurement

4.3 Module Design

SMARTFLOW Functional Modules

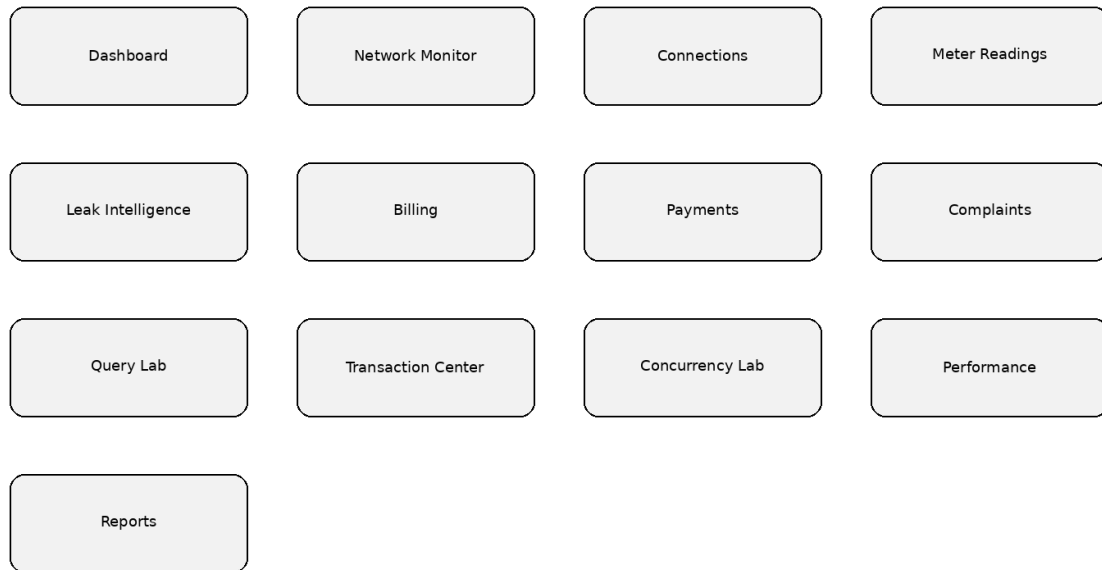


Figure 4.1: SMARTFLOW functional module map.

4.4 Data Flow Design

Operational data flows from users and meter telemetry into the application layer. Validated requests are converted into parameterized SQL. The database returns rows or transaction outcomes, and the application converts them into dashboards, tables, alerts, receipts, and lab visualizations.

4.5 Use Case Design

Actor	Representative Use Cases
Admin	Manage users, seed data, inspect audits, access all DBMS labs
Billing Operator	Generate bills, post payments, reconcile balances
Field Engineer	Inspect readings, investigate leaks, update field status
Complaint Operator	Create, assign, progress, and close complaints
System / Database	Enforce constraints, locks, indexes, transactions, recovery

4.6 ER Design

SMARTFLOW Simplified ER Design

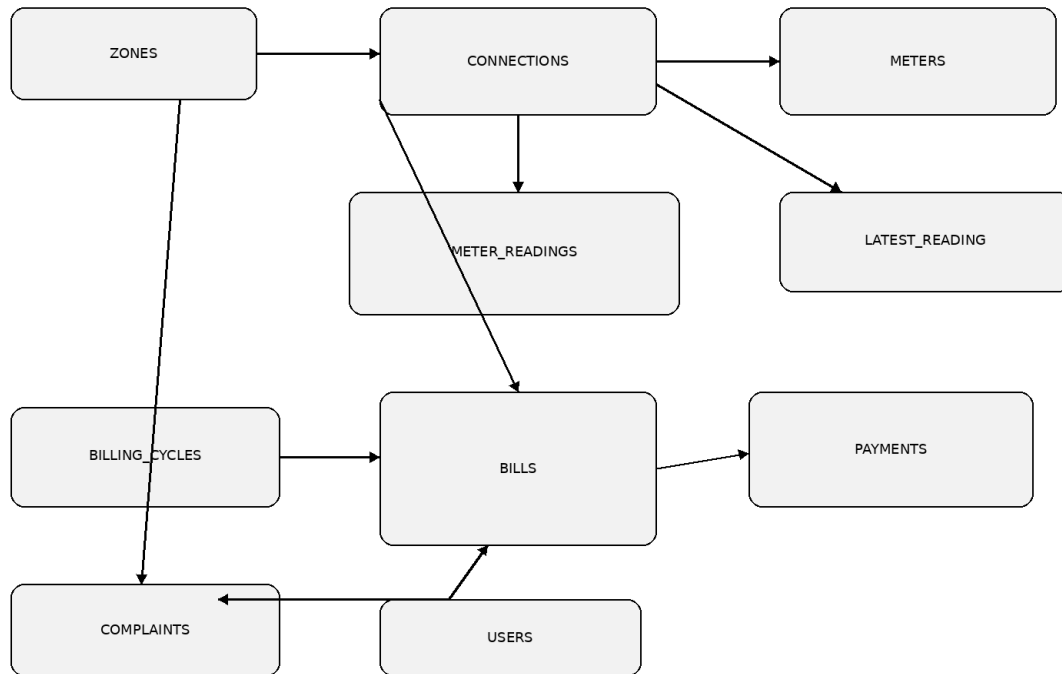


Figure 4.2: Simplified SMARTFLOW entity relationship design.

4.7 Security Design

Security is designed around role-based access, password hashing, parameterized SQL, environment-based database credentials, and audit logging. Client bundles must not contain database credentials.

4.8 Summary

The design translates the requirements into modules, data flows, actors, and a layered architecture.

CHAPTER 5: DATABASE DESIGN

5.1 Database Overview

The proposed database is named smartflow_db and contains 13 normalized tables: users, zones, connections, meters, meter_readings, latest_meter_reading, billing_cycles, bills, bill_items, payments, complaints, complaint_history, and audit_logs.

5.2 Normalization Strategy

The schema separates entities according to their responsibilities and uses primary and foreign keys to represent relationships. Repeating groups and multi-valued operational data are stored in dedicated tables such as bill_items and complaint_history.

5.3 Core Tables

Table	Purpose
users	System actors and roles
zones	Municipal distribution zones
connections	Customer water service endpoints
meters	Smart meters assigned to connections
meter_readings	High-volume telemetry
latest_meter_reading	Current reading per connection
billing_cycles	Monthly billing periods
bills	Utility invoices
bill_items	Tariff line-item breakdown
payments	Payment settlement ledger
complaints	Citizen service tickets
complaint_history	Complaint status audit trail
audit_logs	System-wide forensic activity log

5.4 Primary and Foreign Keys

Primary keys uniquely identify rows. Foreign keys maintain referential integrity between zones and connections, connections and meters/readings, billing cycles and bills, bills and payments, complaints and their histories, and users involved in controlled operations.

5.5 Constraints

- UNIQUE(account_number) for customer accounts.

- UNIQUE(connection_id, cycle_id) for duplicate-bill prevention.
- UNIQUE(transaction_reference) for payment idempotency.
- CHECK(amount > 0) for payments.
- CHECK constraints for valid billing cycle month/year values.
- Foreign keys with deliberate DELETE behavior.

5.6 Database Schema Evidence

Before final submission, capture an actual MySQL Workbench or CLI screenshot showing all 13 created tables and their relationships. The screenshot should be inserted here as Figure 5.x and should not be simulated.

5.7 Summary

The database design provides the structural foundation for all later DBMS demonstrations.

CHAPTER 6: FILE ORGANIZATION, INDEXING AND HASHING

6.1 File Organization Strategy

The telemetry workload is treated as an indexed-sequential relational organization. InnoDB stores the table through B+Tree structures. The design uses a monotonic `reading_id` primary key while adding a composite secondary index for connection-and-time access.

6.2 Sequential vs Hashed vs Indexed-Sequential

Criterion	Sequential	Direct / Hashed	Indexed-Sequential
Point Lookup	$O(N)$	$O(1)$ expected	$O(\log_B N)$
Time Range Query	Poor	Poor	$O(\log_B N + K)$
Temporal Locality	Good globally	Poor	Strong per connection
Collision / Overflow	Not applicable	Possible	Tree page splits
Telemetry Suitability	Limited	Poor for ranges	Strong

6.3 Composite Index

The primary telemetry index is `idx_readings_conn_time` on (`connection_id`, `reading_timestamp` DESC). It is intended to support equality on `connection_id` followed by a timestamp range. This is a SARGable pattern when the timestamp column is compared directly rather than wrapped in a function.

Composite B+Tree Index Access Path

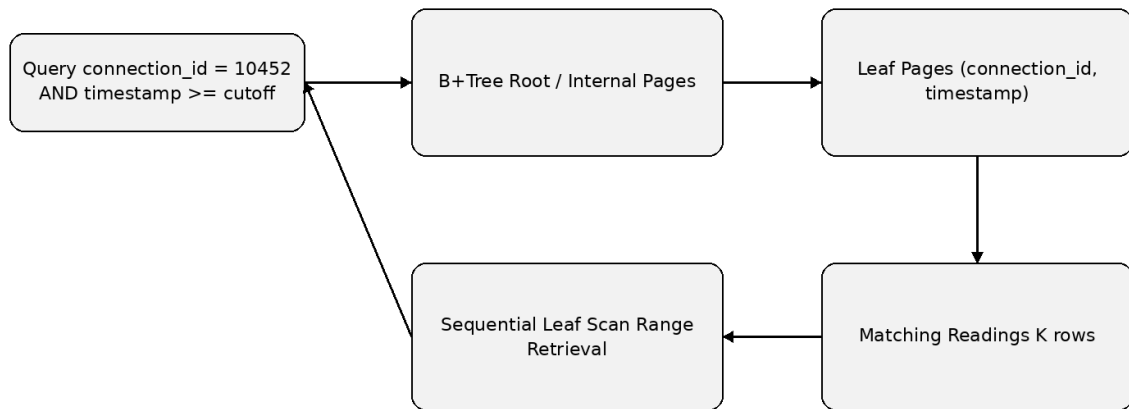


Figure 6.1: Conceptual B+Tree access path for a connection history query.

6.4 Covering Zone Index

The proposed `idx_readings_zone_time_cons` index contains `zone_id`, `reading_timestamp`, and `consumption_liters`. It is intended to support zone-level telemetry scans and aggregate analysis with fewer clustered-row accesses when the query can be satisfied from the index.

6.5 Application-Level Hash Table

The academic hash module is explicitly separate from MySQL indexing. It uses $h(K) = K \bmod N$ with N chosen as a prime bucket count such as 1031. Collisions are resolved through separate chaining. Rehashing occurs when the load factor exceeds 0.75.

Application-Level Hash Table with Separate Chaining

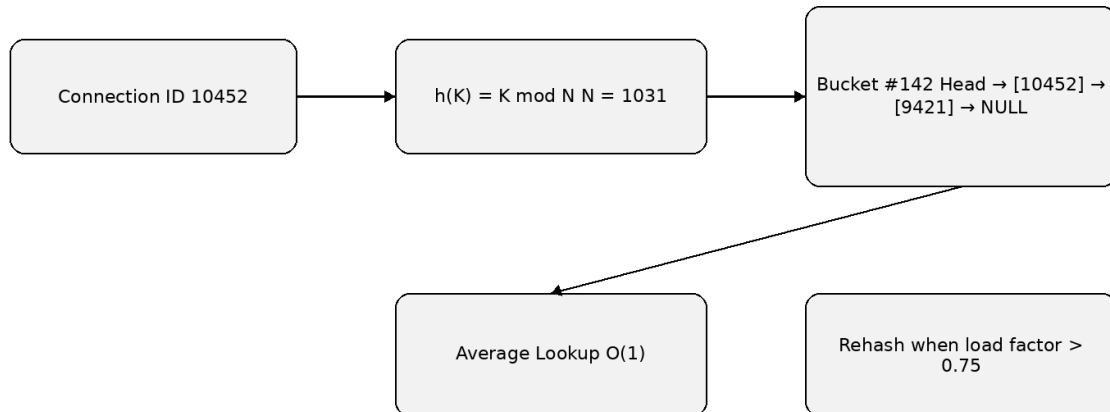


Figure 6.2: Application-level modulo hash with separate chaining.

6.6 Complexity Discussion

Hash lookup is expected $O(1)$ when the load factor and distribution remain controlled. B+Tree lookup is $O(\log_B N)$, while a connection-specific time range is $O(\log_B N + K)$, where K is the number of returned readings.

6.7 Summary

This chapter establishes a defensible academic distinction between relational B+Tree indexing and application-level hashing.

CHAPTER 7: TRANSACTION MANAGEMENT AND CONCURRENCY CONTROL

7.1 ACID Principles

Property	SMARTFLOW Demonstration
Atomicity	Multi-table bill/payment changes commit together or roll back
Consistency	Keys, checks, unique constraints, and business rules protect valid states
Isolation	InnoDB row locking and isolation levels control concurrent access
Durability	Committed changes rely on InnoDB recovery mechanisms

7.2 Billing Transaction

Bill generation begins a transaction, locks the target connection, checks the billing cycle, calculates consumption, inserts the bill and line items, records an audit event, and commits. A duplicate attempt is rejected by the unique connection/cycle invariant.

7.3 Payment Transaction

Payment posting locks the target bill row, validates the outstanding amount, inserts the immutable payment ledger entry, updates paid_amount and due_amount, changes status, and commits. A failure triggers rollback.

7.4 Savepoints

Savepoints provide a partial rollback point inside a larger transaction. SMARTFLOW uses a savepoint before sensitive sub-operations so a failed step can be undone without necessarily discarding the entire parent transaction when the workflow permits it.

ACID Transaction Lifecycle

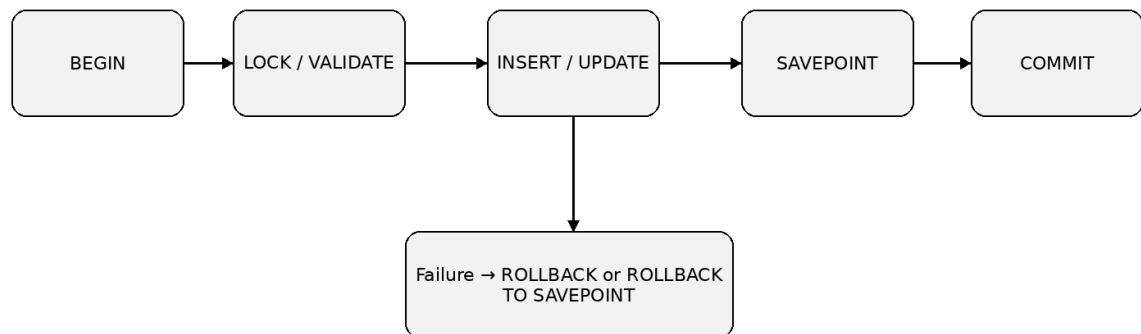


Figure 7.1: ACID transaction lifecycle with savepoint and rollback paths.

7.5 Pessimistic Concurrency Control

`SELECT ... FOR UPDATE` is used where a workflow needs to serialize updates to the same row. This is particularly relevant to bill and payment mutations.

7.6 Optimistic Concurrency Control

Complaints contain a version field. An update includes the previously observed version in its `WHERE` clause. If no row is affected, the application reports a state conflict rather than silently overwriting another operator's update.

7.7 Double Billing Race

Concurrency Control — Double Billing Scenario

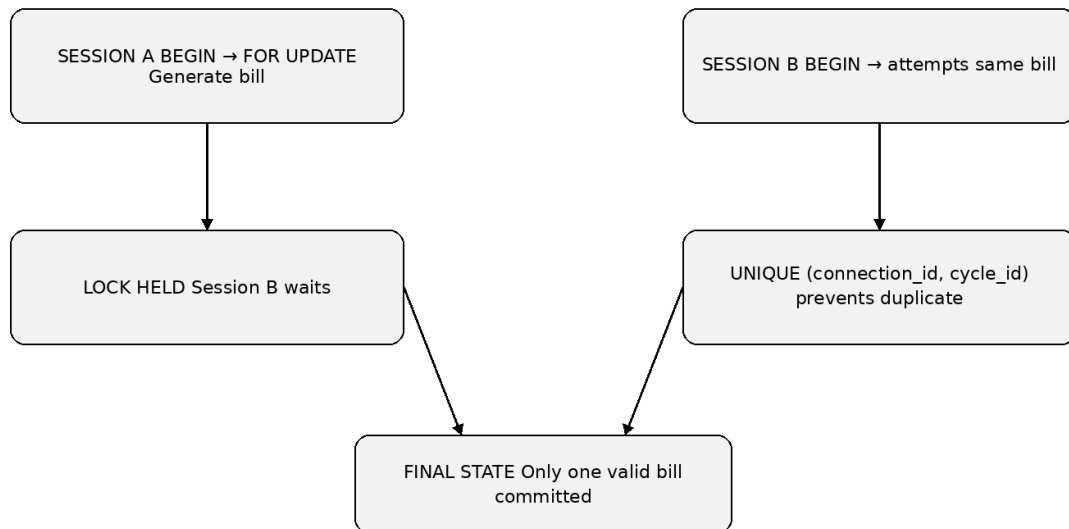


Figure 7.2: Conceptual dual-session double-billing concurrency experiment.

7.8 Summary

The transaction and concurrency design directly connects database theory to operational utility workflows.

CHAPTER 8: LEAK INTELLIGENCE AND ANALYTICS

8.1 Introduction

Leak Intelligence is designed as a statistical anomaly-detection feature rather than as an opaque AI claim. It compares recent consumption with a historical baseline and assigns an operational risk tier.

8.2 Baseline Calculation

For the last M readings, the baseline is the mean historical consumption: $\text{Baseline} = (1/M) \times \sum \text{consumption}_i$. The current deviation is calculated as $((\text{Current} - \text{Baseline}) / \text{Baseline}) \times 100$, subject to protection against a zero baseline.

8.3 Risk Classification

Risk	Deviation / Flow Signal	Action
NORMAL	$\leq +30\%$	Routine monitoring
MODERATE	$+30\%$ to $+75\%$	Customer notification / inspection check
HIGH	$+75\%$ to $+150\%$	Field inspection within operational target
CRITICAL	$> +150\%$ or extreme flow signal	Urgent investigation / shut-off workflow

8.4 Detection Workflow

The workflow retrieves a recent telemetry window, computes a baseline, compares the latest reading, applies the risk rules, and presents the result on the Leak Intelligence page. A critical result can be linked to complaint creation.

8.5 Data Integrity Considerations

The anomaly score is only as reliable as the meter readings. The implementation should therefore distinguish normal readings from reverse-flow, tamper, no-flow, and suspected-leak statuses.

8.6 Summary

The analytics layer provides a transparent rule-based mechanism that can be demonstrated with controlled test data.

CHAPTER 9: QUERY PROCESSING AND OPTIMIZATION

9.1 Query Lab Objective

The Query Lab is designed to make query optimization observable. Each scenario contains a baseline query and an optimized form, followed by live EXPLAIN information and execution measurements.

9.2 Query Scenarios

Scenario	Optimization Focus
Latest reading	Materialized latest-reading lookup / indexed access
Six-month history	Composite connection + timestamp index
Zone leak scan	Zone/time/consumption covering index
Outstanding reconciliation	Indexed joins and pre-aggregation
Payment settlement	Time and payment-method access path
Complaint triage	Zone/status composite index

9.3 SARGability

Predicates should be written so the indexed column can be compared directly. For example, a timestamp range should avoid applying DATE() or another function to the indexed timestamp when that transformation prevents efficient index range access.

9.4 EXPLAIN Evidence

The final report should contain actual EXPLAIN or EXPLAIN ANALYZE screenshots from the implemented database. Expected access types such as ALL, ref, range, or const must be reported from observed output, not assumed in advance.

9.5 Benchmark Metrics

- Wall-clock query latency using a high-resolution timer.
- Access type from EXPLAIN output.
- Rows examined or equivalent optimizer statistics.
- Query throughput under controlled concurrency.
- Comparison across multiple dataset sizes.

9.6 Summary

The Query Lab turns query optimization into a measurable academic experiment.

CHAPTER 10: IMPLEMENTATION

10.1 Implementation Status

The supplied SMARTFLOW material is an implementation plan and architecture specification. It describes the intended React, Node.js, TypeScript, mysql2, and MySQL implementation, but it does not by itself prove that every listed module has been executed successfully. Therefore, this chapter documents the implementation structure while leaving execution evidence to be inserted after the system is actually run.

10.2 Project Structure

```
smartflow/  
  frontend/  
  backend/  
  database/  
  docs/  
  .env.example  
  package.json  
  README.md
```

10.3 Database Scripts

The planned database layer is organized into SQL scripts covering schema creation, constraints and indexes, seed data, domain operations, DBMS lab scenarios, and benchmark support. The final repository should contain executable scripts rather than only documentation.

10.4 Backend Services

The backend is planned around an Express API, mysql2 connection pool, repository/data-access layer, authentication and RBAC middleware, transaction coordinators, hash engine, query profiler, concurrency simulator, and benchmark runner.

10.5 Frontend Implementation

The frontend is planned as a React application with a civic-tech command-center theme. It should provide data-dense tables, KPI cards, charts, modals, filters, toast notifications, and responsive layouts while keeping headings and labels readable.

10.6 Academic Demo Mode

The proposed guided demo contains scenarios for schema/constraints, indexing, hashing, leak detection, billing protection, transactions, concurrency, benchmarking, and auditability. These should be connected to real system actions before being described as live.

10.7 Screenshots to Insert

Evidence	Required Actual Output
Database schema	MySQL Workbench/CLI with 13 tables
Index proof	SHOW INDEX output
Query plans	Before/after EXPLAIN output
Transactions	Actual commit, rollback, savepoint trace
Concurrency	Two real sessions or simulator output
Hashing	Actual collision/chain metrics
Dashboard	Running application screenshot
Reports	Actual generated CSV/print view

10.8 Summary

The implementation chapter is deliberately evidence-aware. It can be finalized after the implementation is run and screenshots/logs are captured.

SMART FLOW

INTELLIGENT WATER UTILITY

OPERATIONS

Overview

Connections

Meter Readings

Leak Intelligence

Billing

Payments

Complaints

DEMS ASSESSMENT LABS

Query Lab (EXPLAIN)

Transaction Center

Concurrency Lab

Performance Lab

Reports & CSV

SIMULATED ASSESSMENT ROLE

ADMIN (Full Access)

CLAS DEMO Assessment

V1.0.0

Zone: All Zones (Metropolitan Wide)

Search Account # or Customer...

Guided Demo Tour (Viva)

MySQL 8.0 • 13 Tables Online

Query Optimization Lab & EXPLAIN Engine

MySQL 8.0 Planner

2. Six-Month Connection History

Run

CONCEPT UNDER TEST

Six-Month Connection Consumption History

SARGable Range Predicate vs. function-Wrapped Non-SARGable DATED Expression

ACCESS TRANSITION: ref → range

ROWS SAVED: -6413 rows

SPEEDUP: -26.4% faster

BASLINE (UNOPTIMIZED)

Anti-Pattern Implementation

ref (B+Tree Non-unique)

SQL STATEMENT

SELECT reading_id, connection_id, reading_timestamp, consumption_liters
FROM meter_readings
WHERE connection_id = 1001
AND DATE(reading_timestamp) >= '2024-03-01'
ORDER BY reading_timestamp DESC;

Identified Anti-Pattern:

Wrapping indexed column in DATE() function suppresses B+Tree range indexing, forcing full scan or suboptimal filtering across 4,416 hourly rows.

KEY CHOSEN

idx_readings_conn_time

ROWS EXAMINED

4,416

FILTERS/OPT OCCURRED

NO

MEASURED TIME

19.785 ms

EXTRA:

NONE

OPTIMIZED QUERY

Indexed & SARGable Implementation

range (B+Tree Interval)

SQL STATEMENT

SELECT reading_id, connection_id, reading_timestamp, consumption_liters
FROM meter_readings FORCE INDEX (idx_readings_conn_time)
WHERE connection_id = 1001
AND reading_timestamp >= '2024-03-01 00:00:00'
ORDER BY reading_timestamp DESC;

Applied Optimization:

SARGable ISO timestamp range utilizing Secondary B+Tree Composite Index (connection_id, reading_timestamp DESC). Access type range; zero filesort.

KEY CHOSEN

idx_readings_conn_time

ROWS EXAMINED

1

COVERING INDEX

NO

MEASURED TIME

25.856 ms

EXTRA:

NONE

ACID Transaction Center & State Machine

InnoDB Atomicity

State: Idle

TRANSACTION CONTROLS

1. START TRANSACTION

2. SELECT ... FOR UPDATE

3. SAVEPOINT sp_checkpoint_1

4. UPDATE due_amount (-F50.00)

5. ROLLBACK TO SAVEPOINT

COMMIT

ROLLBACK

TARGET LEADER STATE: ROLL #2002

Total Billable:

\$299.25

Uncommitted Due:

\$240.25

MySQL 8.0 InnoDB Transaction Journal Log

Clear Log

[SYSTEM] Transaction Sandbox initialized.
[SYSTEM] Connected to MySQL 8.0 (InnoDB Engine, REPEATABLE READ ISOLATION).
[20:51:26] SQL: START TRANSACTION;
[20:51:26] InnoDB: transaction boundary opened. Read view established. Undo logs initialized.
[20:51:26] SQL: COMMIT;
[20:51:26] InnoDB: Transaction committed atomically. Redo log flushed to disk. All locks released.

A - ATOMICITY
All-or-nothing execution

C - CONSISTENCY
CHECK & FK preserved

I - ISOLATION
FOR UPDATE row locking

D - DURABILITY
Redo log write to disk

CHAPTER 11: TESTING, RESULTS AND DISCUSSION

11.1 Testing Strategy

Testing should cover constraints, transactions, concurrency, indexing, hashing, leak analytics, security, and user workflows. Each test should record the input, observed output, pass/fail status, and evidence file.

11.2 Database Tests

Test ID	Scenario	Expected Result
TC-DB-01	Duplicate bill	Second insert rejected by unique constraint
TC-DB-02	Negative payment	Rejected by CHECK constraint
TC-TX-01	Bill item failure	Transaction rolls back
TC-TX-02	Savepoint rollback	Sub-operation restored while parent continues where valid
TC-CC-01	Concurrent billing	Only one bill committed
TC-CC-02	Concurrent payment/update	Row locking prevents lost update
TC-IDX-01	History query	Index-based range access observed
TC-HSH-01	Hash lookup	Lookup returns same latest state as DB
TC-LEAK-01	300% deviation	CRITICAL classification

11.3 Results Reporting

Results must be reported from actual executions. If a test has not been executed, its status should remain Pending Execution. The report should never convert planned measurements into fabricated results.

11.4 Performance Discussion

The expected experimental comparison is between an unoptimized scan, an indexed query, and an application-level hash lookup. The final conclusion should be based on measured latency and access plans on the chosen machine and dataset.

11.5 Usability Discussion

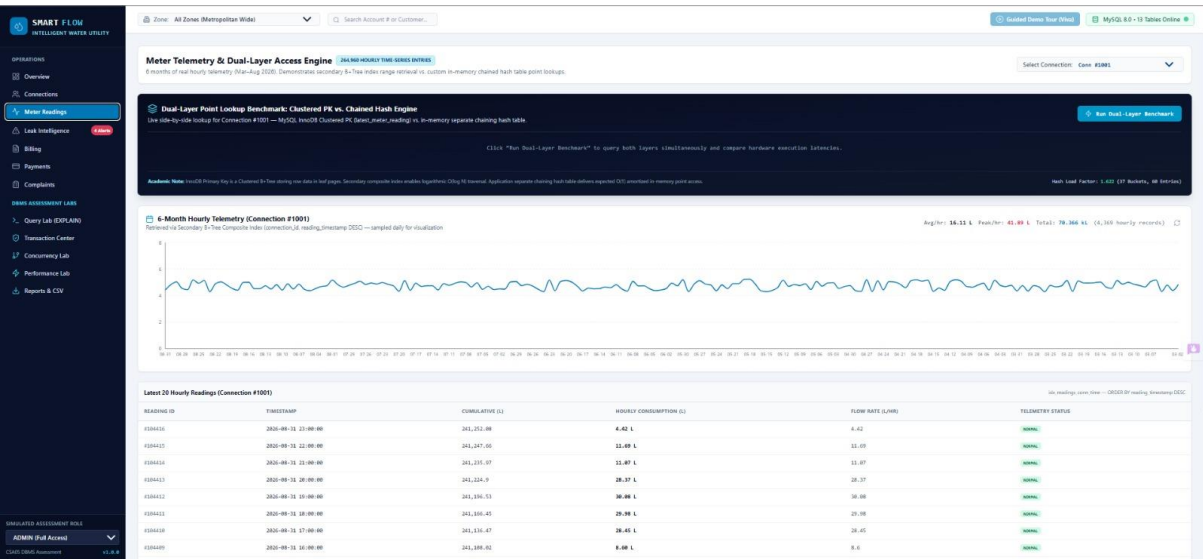
The interface should minimize cognitive load through consistent navigation, clear hierarchy, readable tables, descriptive empty states, loading skeletons, and visible success/error feedback.

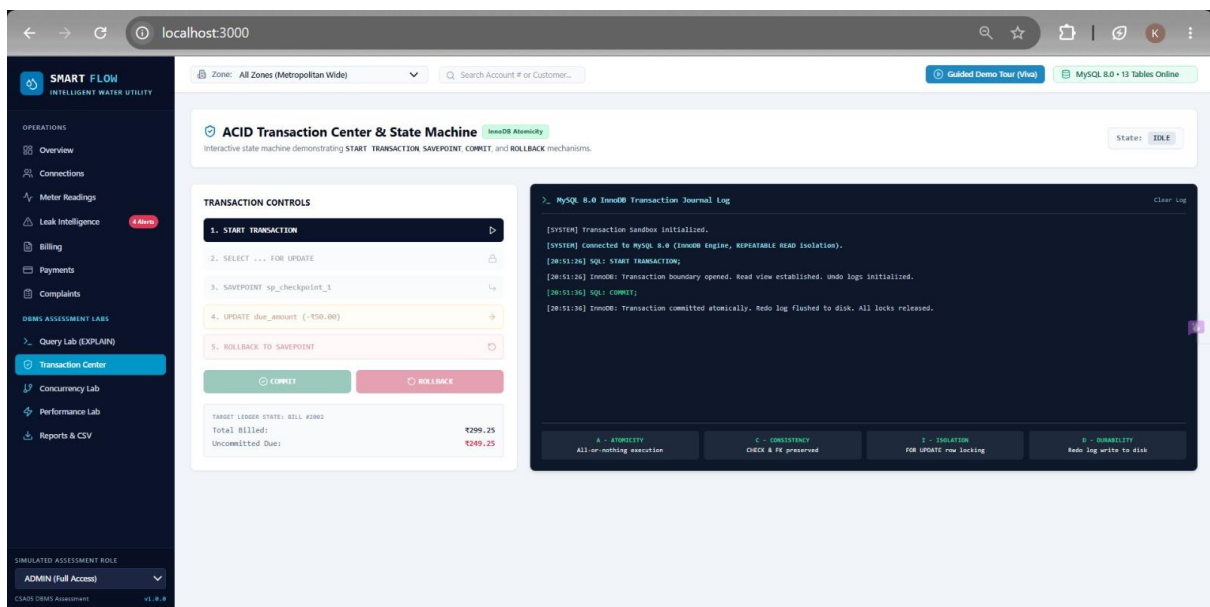
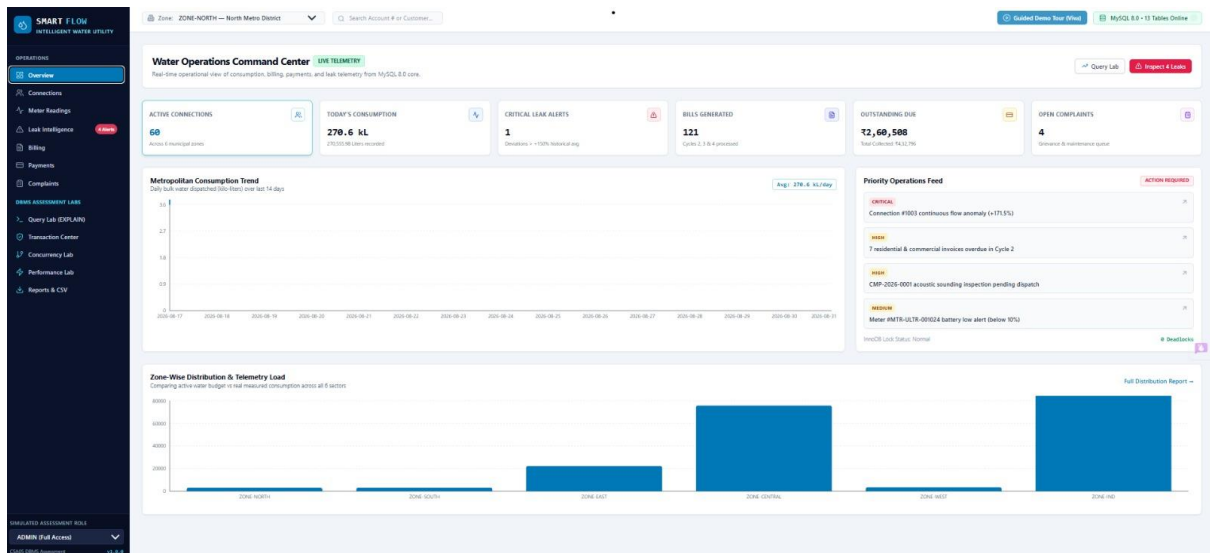
11.6 Limitations

Benchmark results can vary by hardware, buffer pool state, dataset distribution, cache warmth, concurrent load, and MySQL configuration. Application-level hash performance should also be interpreted separately from durable database indexing because the hash table resides in application memory.

11.7 Summary

This chapter provides a defensible framework for converting implementation output into final academic results.





CHAPTER 12: SECURITY, CONCLUSION AND FUTURE ENHANCEMENTS

12.1 Security Architecture

- Parameterized SQL statements through mysql2 placeholders.
- Environment variables for database credentials.
- Password hashing rather than plaintext storage.
- Role-based authorization for protected modules.
- Audit records for critical financial and operational mutations.
- Validation of user input before database operations.

12.2 Auditability

The `audit_logs` table is intended to provide forensic visibility into billing, payment, complaint, and rollback actions. Audit records should contain the acting user, entity, action type, details, and timestamp.

12.3 Conclusion

SMARTFLOW presents a comprehensive Database Management Systems capstone design for intelligent municipal water utility management. The project combines a normalized relational schema with high-volume telemetry handling, composite B+Tree indexing, application-level hashing, statistical leak detection, transaction processing, concurrency control, query optimization, benchmarking, and role-based security.

The major academic strength of the design is that each DBMS concept is tied to a concrete utility workflow. Billing demonstrates uniqueness and transaction safety; payments demonstrate atomic balance reconciliation; complaints demonstrate optimistic concurrency; telemetry demonstrates indexing and range access; Query Lab demonstrates execution plans; and the Performance module demonstrates measurement.

12.4 Future Enhancements

- Real-time streaming ingestion from smart meters.
- Message queues for high-volume telemetry ingestion.
- Advanced statistical or machine-learning leak prediction.
- GIS-based network visualization.
- SMS/email notification integration.

- Mobile applications for field engineers and customers.
- Horizontal scaling and partitioning for very large telemetry volumes.
- Advanced observability with metrics and distributed tracing.

12.5 Final Statement

SMARTFLOW is intended to demonstrate not only a functional utility-management interface but also the reasoning behind database organization, indexing, transaction safety, concurrency control, and performance. Final claims should be supported by actual execution evidence collected from the completed implementation.

SMART FLOW

INTELLIGENT WATER UTILITY

OPERATIONS

Overview

Connections

Meter Readings

Leak Intelligence

Billing

Payments

Complaints

DBMS ASSESSMENT LABS

Query Lab (EXPLAIN)

Transaction Center

Concurrency Lab

Performance Lab

Reports & CSV

SIMULATED ASSESSMENT ROLE

ADMIN (Full Access)

CSMS DBMS Assessment

v1.0.0

Zone: All Zones (Metropolitan Wide)

Search Account # or Customer...

Guided Demo Tour (Viva)

MySQL 8.0 - 13 Tables Online

Complaints Triage & Work Orders

Optimistic Concurrency Control

Citizen grievances and field work orders protected against lost updates using monotonic version tokens.

Active Grievances: 6

All Lifecycle Statuses

All Priorities

TICKET #	CATEGORY	PRIORITY	CUSTOMER & ZONE	ASSIGNED CREW	STATUS	OCC VERSION	ACTIONS
CMP-2026-0006	LEAK	CRITICAL	Kaveri Infotech Park ACC-COP-001082 (ZONE-NORTH)	Unassigned	OPEN	v1	Triage --
CMP-2026-0001	LEAK	CRITICAL	Ajrun Chatterjee ACC-009-001081 (ZONE-SOUTH)	Suresh Kulkarni (Hydraulic Engineer South)	ASSIGNED	v1	Triage --
CMP-2026-0002	LEAK	HIGH	Jagdish Hegde ACC-009-001083 (ZONE-CENTRAL)	Vikram Desai (Senior Field Engineer)	IN PROGRESS	v2	Triage --
CMP-2026-0003	METER_FAULT	MEDIUM	Pranav Chauhan ACC-009-001084 (ZONE-SOUTH)	Unassigned	OPEN	v1	Triage --
CMP-2026-0004	LOW_PRESSURE	MEDIUM	Deepa Chauhan ACC-009-001086 (ZONE-NORTH)	Vikram Desai (Senior Field Engineer)	RESOLVED	v3	Triage --
CMP-2026-0005	BILLING_DISPUTE	LOW	Geeta Kulkarni ACC-009-001082 (ZONE-NORTH)	Priya Ramanathan (Chief Revenue Officer)	CLOSED	v2	Triage --

Page 1 of 1 (6 total tickets)

Previous

Next

REFERENCES

8. Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts. McGraw-Hill.
9. Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems. Pearson.
10. Oracle Corporation. MySQL 8.0/8.4 Reference Manual.
11. Node.js Documentation. Node.js Runtime and API Documentation.
12. Express.js Documentation. Express Web Framework Documentation.
13. React Documentation. React User Interface Library Documentation.
14. TypeScript Documentation. TypeScript Language Handbook.
15. Tailwind CSS Documentation. Utility-First CSS Framework Documentation.
16. Recharts Documentation. React Charting Library Documentation.

Zone: All Zones (Metropolitan Wide) Search Account # or Customer...

Guided Demo Tour (Viva) MySQL 8.0 • 13 Tables Online

Billing Management & Invoice Engine ACID Transaction Safe [Generate New Bill](#)

Automated tiered slab tariff calculation with strict unique constraint duplicate-bill prevention.

All Billing Cycles All Invoice Statuses Showing 15 of 128 Invoices

BILL NUMBER	CUSTOMER & ACCOUNT	ZONE	CYCLE	CONSUMED	TOTAL AMOUNT	PAID AMOUNT	BALANCE DUE	STATUS	INVOICE
INV-202608-C1060	State Police Training Academy ACC-130-001300	ZONE-EAST	CYC-2026-08	102 KL	₹1979.25	₹589.63	₹589.62	PARTIALLY PAID	View --
INV-202607-C1060	State Police Training Academy ACC-130-001300	ZONE-EAST	CYC-2026-07	95 KL	₹1850.63	₹1850.63	₹0.00	PAID	View --
INV-202608-C1059	Metropolitan Public Library & Archives ACC-130-001309	ZONE-EAST	CYC-2026-08	102 KL	₹1979.25	₹1979.25	₹0.00	PAID	View --
INV-202607-C1059	Metropolitan Public Library & Archives ACC-130-001309	ZONE-EAST	CYC-2026-07	95 KL	₹1850.63	₹1850.63	₹0.00	PAID	View --
INV-202608-C1058	Government Model Higher Secondary School ACC-130-001308	ZONE-EAST	CYC-2026-08	102 KL	₹1979.25	₹0.00	₹1979.25	GENERATED	View --
INV-202607-C1058	Government Model Higher Secondary School ACC-130-001308	ZONE-EAST	CYC-2026-07	95 KL	₹1850.63	₹1850.63	₹0.00	PAID	View --
INV-202608-C1057	St. Jude Educational Campus ACC-130-001307	ZONE-EAST	CYC-2026-08	102 KL	₹1979.25	₹0.00	₹1979.25	GENERATED	View --
INV-202607-C1057	St. Jude Educational Campus ACC-130-001307	ZONE-EAST	CYC-2026-07	95 KL	₹1850.63	₹1850.63	₹0.00	PAID	View --
INV-202608-C1056	Indus Heavy Machinery Works ACC-130-001306	ZONE-IND	CYC-2026-08	700 KL	₹46593.75	₹46593.75	₹0.00	PAID	View --

APPENDICES

Appendix A – Complete Table List

No.	Table
1	users
2	zones
3	connections
4	meters
5	meter_readings
6	latest_meter_reading
7	billing_cycles
8	bills
9	bill_items
10	payments
11	complaints
12	complaint_history
13	audit_logs

Appendix B – Key SQL Patterns

```
SELECT connection_id, reading_timestamp, consumption_liters FROM meter_readings
WHERE connection_id = ? AND reading_timestamp >= ? ORDER BY reading_timestamp
DESC;
```

```
SELECT connection_id, cycle_id FROM bills WHERE connection_id = ? AND cycle_id
= ? FOR UPDATE;
```

```
SELECT bill_id, total_amount, paid_amount, due_amount FROM bills WHERE bill_id
= ? FOR UPDATE;
```

```
UPDATE complaints SET status = ?, version = version + 1 WHERE complaint_id = ?
AND version = ?;
```

Appendix C – Evidence and Screenshot Checklist

No.	Evidence
1	Database schema and foreign keys
2	Constraints / duplicate rejection
3	Composite index output
4	Unindexed EXPLAIN
5	Indexed EXPLAIN
6	Hash comparison

7	Transaction commit
8	Transaction rollback
9	Savepoint
10	Concurrency race
11	Dashboard
12	Connection 360 view
13	Telemetry table
14	Leak Intelligence
15	Billing invoice
16	Payment receipt
17	Complaint board
18	Query Lab
19	Transaction Center
20	Concurrency Lab
21	Performance benchmark
22	Audit trail
23	CSV export
24	RBAC restriction
25	Guided demo scenario

Appendix D – Figure Index

Figure	Title
3.1	SMARTFLOW three-layer architecture
3.2	Operational workflow
4.1	Functional module map
4.2	Simplified ER design
6.1	B+Tree access path
6.2	Application-level hashing
7.1	ACID transaction lifecycle
7.2	Concurrency experiment

Appendix E – Report Finalization Notes

Replace every placeholder for student names, guide, institution, dates, screenshots, execution plans, measured benchmarks, and actual test results before submission. Update the Table of Contents after final formatting in Microsoft Word.

